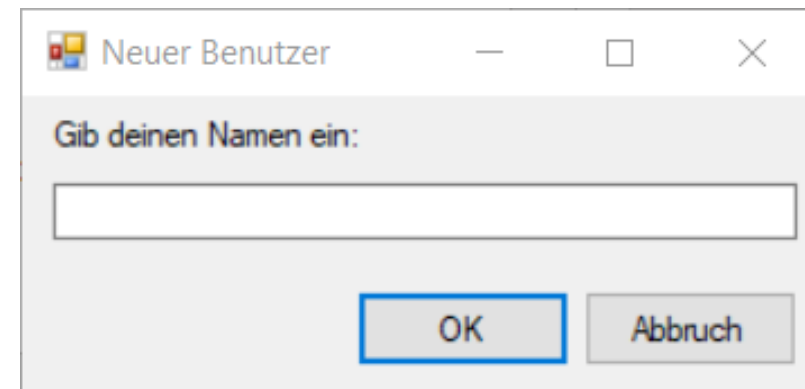


C# mit GUI

am Beispiel von Windows Forms

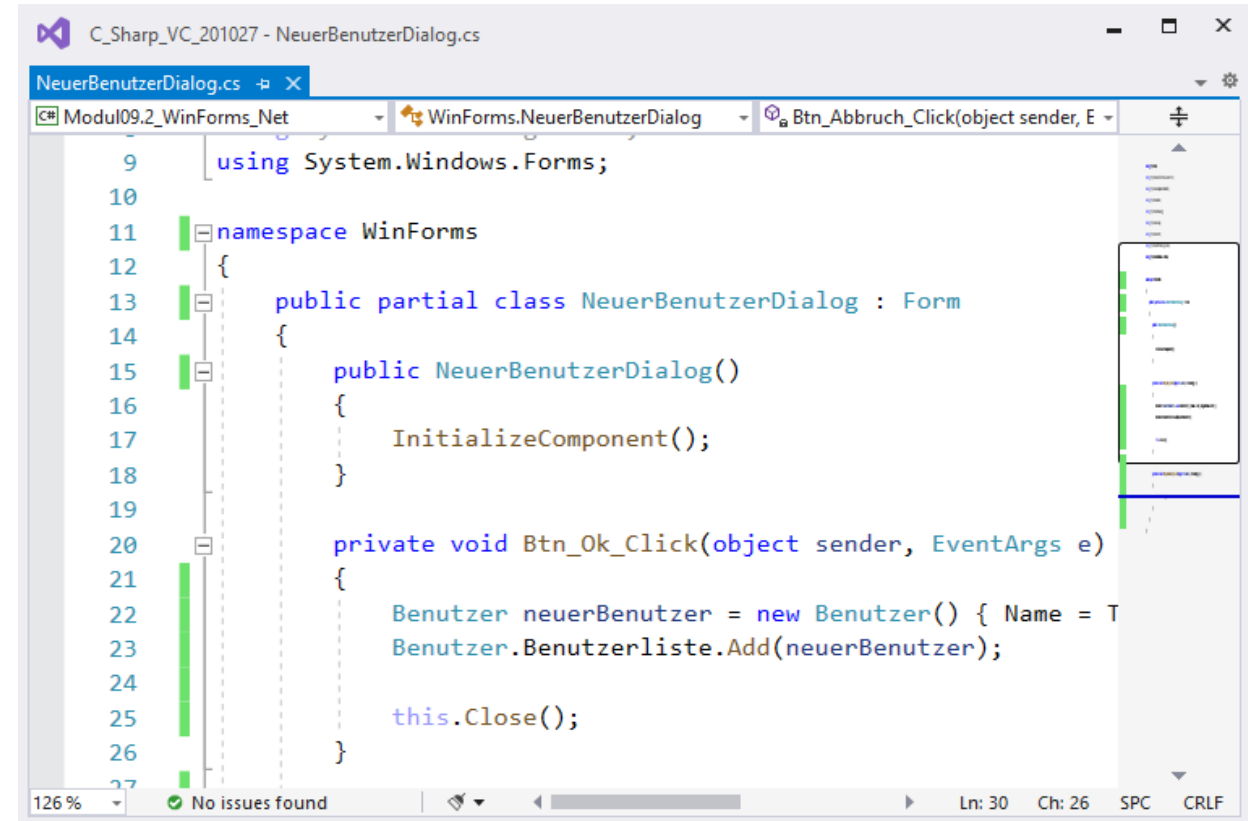
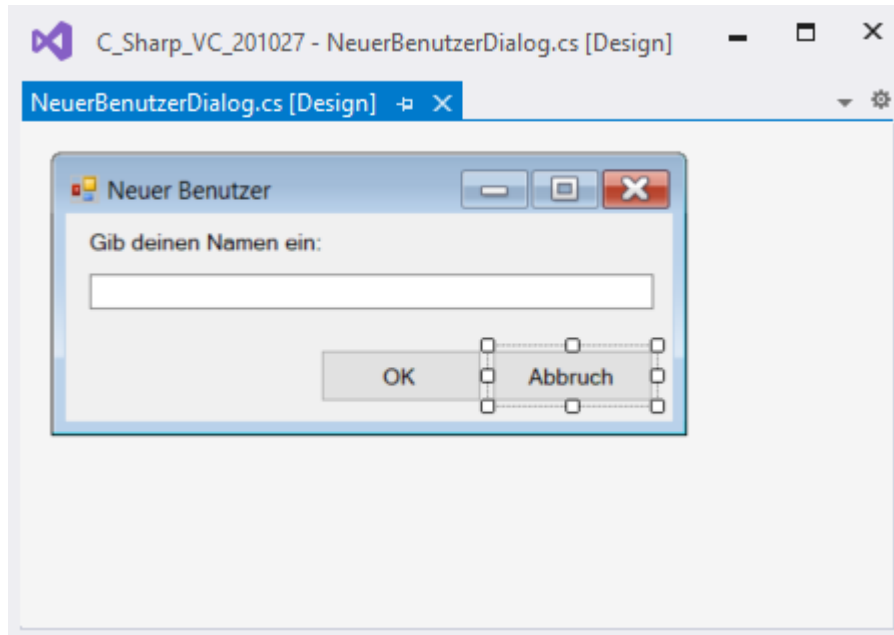
Windows Forms

- Windows Forms ist das ‚alte‘ UI-Framework für Desktop-Programme
 - ‚neues‘ UI-Framework: Windows Presentation Foundation (WPF)
- Form: Klasse zur Darstellung eines Windows-Fensters
- Event-basierte Userinteraktion
- Design mittels visuellem Designer



Windows Forms

- ‚zweigeteilte‘ Bearbeitung:
 - GUI-Design im visuellen Designer
 - Logik im ‚Code-Behind‘

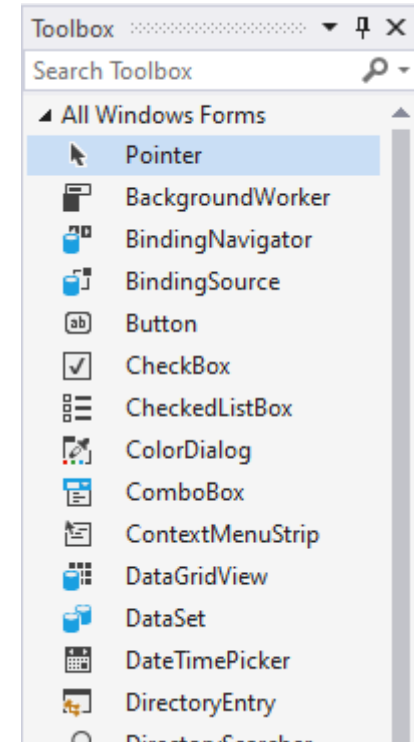
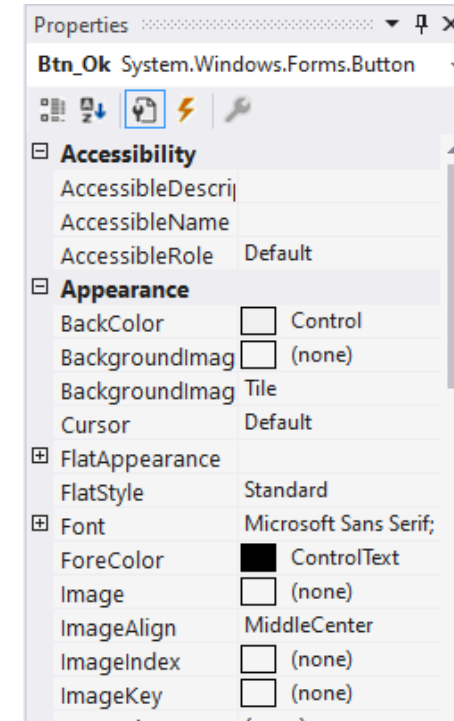


partielle Klassen und Methoden

- Klassen und Methoden können auch in unterschiedlichen Quelldateien aufgeteilt werden
- jede Quelldatei enthält Teile der Typ- oder Methodendefinition
- werden bei der Kompilierung der Anwendung kombiniert
- jede Teilklasse muss als partial markiert werden
- partielle Methoden können nur den Rückgabewert „void“ haben
- Schlüsselwort „partial“ vor dem Schlüsselwort „class“ oder „void“ stehen
- Methoden müssen privat sein und können nur einmal implementiert werden

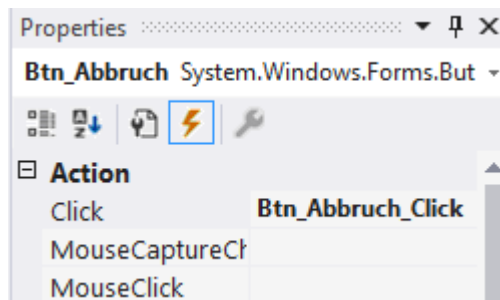
Windows Forms mit Visual Studio

- ‚Toolbox‘ beinhaltet die GUI-Objekte
 - Drag’n’Drop zum Plazieren im Designer
- ‚Eigenschaften‘-Fenster gewährt Zugriff auf die Properties des merkierten GUI-Objekts



Eigenschaften und Events

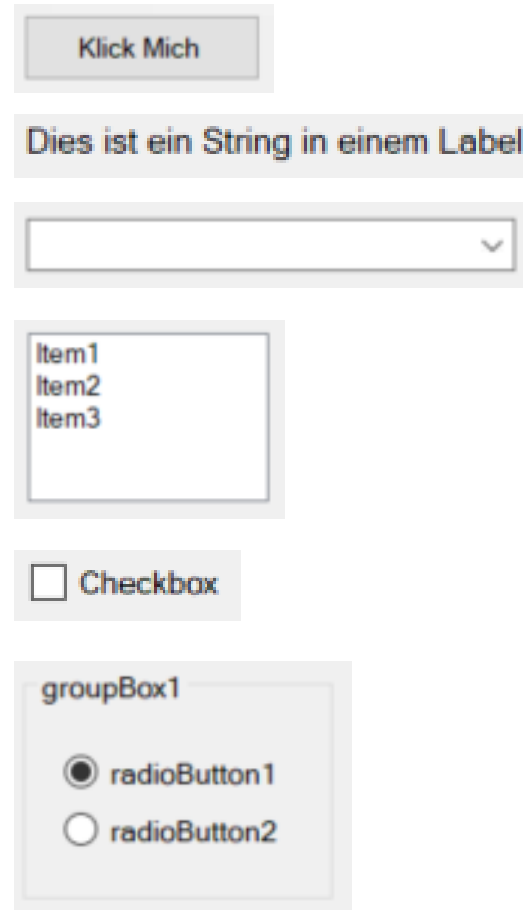
- Im ‚Eigenschaften‘- Fenster () findet sich eine Auflistung aller zugänglichen Eigenschaften des im Designer markierten Objekts
 - (Name) definiert den Variablen-Bezeichner, unter dem das Objekt aufgerufen werden kann
- Im ‚Events‘-Fenster () befindet sich eine Liste der verfügbaren Events
 - Events müssen mit einer EventHandler-Methode im Code-Behind verknüpft sein



```
private void Btn_Abbruch_Click(object sender, EventArgs e)
{
    this.Close();
}
```

Wichtige GUI-Objekte

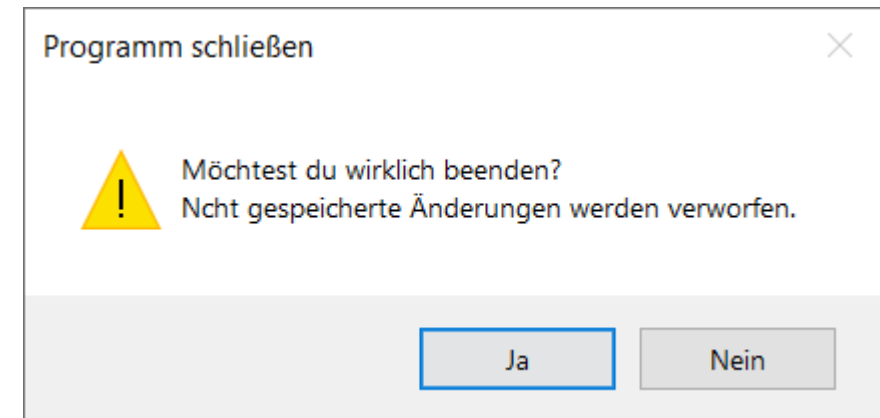
- Button
- Label
- ComboBox
- ListBox
- CheckBox
- RadioButton



- GUI-Klassen erben (indirekt) von `System.Windows.Forms.Control`
- Neben den Ererbten hat jede GUI-Klasse spezifische, klasseneigene Eigenschaften und Events

MessageBox

- Message-Boxen sind kleine, automatisch generierte Dialogfenster
- Für User-Nachfragen oder Informationen
- Aufruf erfolgt über statische Methode `MessageBox.Show()`
- Rückgabe vom Enum-Typ `DialogResult` entspricht geklicktem Button



```
string dialogText = "Möchtest du wirklich beenden?\nNicht gespeicherte Änderungen werden verworfen.";
DialogResult result = MessageBox.Show(dialogText, "Programm schließen", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
if (result == DialogResult.Yes)
    this.Close();
```


Schließen und Öffnen von Forms

- `this.Close()` schließt das aktuelle Form
- `Form.Show()` öffnet ein Form als gleichberechtigtes Fenster
- `Form.ShowDialog()` öffnet ein Form als Dialog-Fenster
 - Dialogfenster müssen bearbeitet und geschlossen werden, bevor der User ein anderes Fenster bearbeiten kann
 - ShowDialog gibt `DialogResult` zurück